```
# ============================
# PIX2PIX FULL IMPLEMENTATION
# ============================

!pip -q install torchvision pillow tqdm

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.utils as vutils
from torch.utils.data import Dataset, DataLoader, Subset
from PIL import Image
import os
import urllib.request
import tarfile
from tqdm import tqdm
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using:", device)


dataset_url = "https://efrosgans.eecs.berkeley.edu/pix2pix/datasets/edges2shoes.tar.gz"
dataset_file = "edges2shoes.tar.gz"

if not os.path.exists(dataset_file):
    print("Downloading dataset...")
    urllib.request.urlretrieve(dataset_url, dataset_file)

if not os.path.exists("edges2shoes"):
    print("Extracting dataset...")
    with tarfile.open(dataset_file, "r:gz") as tar:
        tar.extractall()

print("Dataset Ready")

# ======================================================
# 2. DATASET CLASS
# ======================================================

class Pix2PixDataset(Dataset):
    def __init__(self, root_dir):
        self.root_dir = root_dir
        self.files = os.listdir(root_dir)

        self.transform = transforms.Compose([
            transforms.Resize((128,128)),   # faster training
            transforms.ToTensor(),
            transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
        ])

    def __len__(self):
        return len(self.files)

    def __getitem__(self, index):
        img_path = os.path.join(self.root_dir, self.files[index])
        img = Image.open(img_path).convert("RGB")

        w, h = img.size
        w2 = w // 2

        edge = img.crop((0,0,w2,h))
        real = img.crop((w2,0,w,h))

        edge = self.transform(edge)
        real = self.transform(real)

        return edge, real

# ======================================================
# 3. DATALOADERS
# ======================================================

train_dataset = Pix2PixDataset("edges2shoes/train")
train_dataset = Subset(train_dataset, range(3000))

train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True, num_workers=2, pin_memory=True)

test_dataset = Pix2PixDataset("edges2shoes/val")
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False, num_workers=2, pin_memory=True)
```

```
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False, num_workers=2, pin_memory=True)

# ======================================================
# 4. U-NET GENERATOR
# ======================================================

class DownBlock(nn.Module):
    def __init__(self, in_c, out_c, normalize=True):
        super().__init__()
        layers = [nn.Conv2d(in_c, out_c, 4, 2, 1, bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_c))
        layers.append(nn.LeakyReLU(0.2))
        self.block = nn.Sequential(*layers)
    def forward(self,x): return self.block(x)

class UpBlock(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.block = nn.Sequential(
            nn.ConvTranspose2d(in_c,out_c,4,2,1,bias=False),
            nn.BatchNorm2d(out_c),
            nn.ReLU(True)
        )
    def forward(self,x,skip):
        x=self.block(x)
        return torch.cat((x,skip),1)

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.d1=DownBlock(3,64,False)
        self.d2=DownBlock(64,128)
        self.d3=DownBlock(128,256)
        self.d4=DownBlock(256,512)
        self.d5=DownBlock(512,512)
        self.d6=DownBlock(512,512)

        self.u1=UpBlock(512,512)
        self.u2=UpBlock(1024,512)
        self.u3=UpBlock(1024,256)
        self.u4=UpBlock(512,128)
        self.u5=UpBlock(256,64)

        self.final=nn.Sequential(
            nn.ConvTranspose2d(128,3,4,2,1),
            nn.Tanh()
        )

    def forward(self,x):
        d1=self.d1(x)
        d2=self.d2(d1)
        d3=self.d3(d2)
        d4=self.d4(d3)
        d5=self.d5(d4)
        d6=self.d6(d5)

        u1=self.u1(d6,d5)
        u2=self.u2(u1,d4)
        u3=self.u3(u2,d3)
        u4=self.u4(u3,d2)
        u5=self.u5(u4,d1)

        return self.final(u5)

# ======================================================
# 5. PATCHGAN DISCRIMINATOR
# ======================================================

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        def block(in_c,out_c,normalize=True):
            layers=[nn.Conv2d(in_c,out_c,4,2,1)]
            if normalize: layers.append(nn.BatchNorm2d(out_c))
            layers.append(nn.LeakyReLU(0.2))
            return layers

        self.model=nn.Sequential(
            *block(6,64,False),
            *block(64,128),
            *block(128,256),
            *block(256,512),
```

```
            nn.Conv2d(512,1,4,1,1)
        )
    def forward(self,edge,real):
        x=torch.cat([edge,real],1)
        return self.model(x)

G=Generator().to(device)
D=Discriminator().to(device)

# ======================================================
# 6. LOSSES & OPTIMIZERS
# ======================================================

criterion_GAN=nn.BCEWithLogitsLoss()
criterion_L1=nn.L1Loss()
lambda_L1=100

opt_G=optim.Adam(G.parameters(),lr=0.0002,betas=(0.5,0.999))
opt_D=optim.Adam(D.parameters(),lr=0.0002,betas=(0.5,0.999))

scaler_G=torch.cuda.amp.GradScaler()
scaler_D=torch.cuda.amp.GradScaler()

# ======================================================
# 7. SAVE OUTPUT IMAGES
# ======================================================

os.makedirs("results",exist_ok=True)

def save_examples(epoch):
    G.eval()
    edge,real=next(iter(test_loader))
    edge=edge.to(device)
    real=real.to(device)
    with torch.no_grad():
        fake=G(edge)
    grid=torch.cat([edge,fake,real],0)
    grid=(grid+1)/2
    vutils.save_image(grid,f"results/epoch_{epoch}.png",nrow=edge.size(0))
    G.train()

# ======================================================
# 8. TRAIN PIX2PIX
# ======================================================

epochs=10

for epoch in range(epochs):
    loop=tqdm(train_loader)
    for i,(edge,real) in enumerate(loop):
        edge=edge.to(device)
        real=real.to(device)

        if i%2==0:
            with torch.cuda.amp.autocast():
                fake=G(edge)
                D_real=D(edge,real)
                D_fake=D(edge,fake.detach())
                loss_real=criterion_GAN(D_real,torch.ones_like(D_real))
                loss_fake=criterion_GAN(D_fake,torch.zeros_like(D_fake))
                loss_D=(loss_real+loss_fake)/2

            opt_D.zero_grad()
            scaler_D.scale(loss_D).backward()
            scaler_D.step(opt_D)
            scaler_D.update()

        with torch.cuda.amp.autocast():
            fake=G(edge)
            D_fake=D(edge,fake)
            loss_GAN=criterion_GAN(D_fake,torch.ones_like(D_fake))
            loss_L1=criterion_L1(fake,real)
            loss_G=loss_GAN+lambda_L1*loss_L1

        opt_G.zero_grad()
        scaler_G.scale(loss_G).backward()
        scaler_G.step(opt_G)
        scaler_G.update()

        loop.set_description(f"Epoch [{epoch+1}/{epochs}]")
        loop.set_postfix(D_loss=loss_D.item(),G_loss=loss_G.item())

    save_examples(epoch+1)
```

```
# ======================================================
# 9. BASELINE CNN (FOR COMPARISON)
# ======================================================

class CNNBaseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder=nn.Sequential(
            nn.Conv2d(3,64,4,2,1),nn.ReLU(),
            nn.Conv2d(64,128,4,2,1),nn.ReLU(),
            nn.Conv2d(128,256,4,2,1),nn.ReLU()
        )
        self.decoder=nn.Sequential(
            nn.ConvTranspose2d(256,128,4,2,1),nn.ReLU(),
            nn.ConvTranspose2d(128,64,4,2,1),nn.ReLU(),
            nn.ConvTranspose2d(64,3,4,2,1),nn.Tanh()
        )
    def forward(self,x):
        return self.decoder(self.encoder(x))

baseline=CNNBaseline().to(device)
opt=optim.Adam(baseline.parameters(),lr=0.0002)
criterion=nn.L1Loss()

for epoch in range(5):
    for edge,real in train_loader:
        edge=edge.to(device)
        real=real.to(device)
        output=baseline(edge)
        loss=criterion(output,real)
        opt.zero_grad()
        loss.backward()
        opt.step()

print("Training Complete. Check /results folder for Pix2Pix outputs.")
```

```
Using: cuda
Downloading dataset...
Extracting dataset...
/tmp/ipython-input-908/1468128023.py:34: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and
  tar.extractall()
Dataset Ready
/tmp/ipython-input-908/1468128023.py:184: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `tor
  scaler_G=torch.cuda.amp.GradScaler()
/tmp/ipython-input-908/1468128023.py:185: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `tor
  scaler_D=torch.cuda.amp.GradScaler()
  0%|          | 0/750 [00:00<?, ?it/s]/tmp/ipython-input-908/1468128023.py:218: FutureWarning: `torch.cuda.amp.autocast(arg
  with torch.cuda.amp.autocast():
/tmp/ipython-input-908/1468128023.py:231: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch
  with torch.cuda.amp.autocast():
Epoch [1/10]: 100%|██████████| 750/750 [00:29<00:00, 25.42it/s, D_loss=0.523, G_loss=29.3]
Epoch [2/10]: 100%|██████████| 750/750 [00:29<00:00, 25.40it/s, D_loss=0.275, G_loss=18.2]
Epoch [3/10]: 100%|██████████| 750/750 [00:28<00:00, 26.60it/s, D_loss=0.44, G_loss=14.7]
Epoch [4/10]: 100%|██████████| 750/750 [00:29<00:00, 25.70it/s, D_loss=0.766, G_loss=27.8]
Epoch [5/10]: 100%|██████████| 750/750 [00:27<00:00, 26.95it/s, D_loss=1.14, G_loss=12.6]
Epoch [6/10]: 100%|██████████| 750/750 [00:28<00:00, 26.72it/s, D_loss=0.343, G_loss=12.9]
Epoch [7/10]: 100%|██████████| 750/750 [00:27<00:00, 27.27it/s, D_loss=0.379, G_loss=12.3]
Epoch [8/10]: 100%|██████████| 750/750 [00:28<00:00, 26.66it/s, D_loss=0.789, G_loss=18.4]
Epoch [9/10]: 100%|██████████| 750/750 [00:28<00:00, 26.38it/s, D_loss=0.596, G_loss=14.8]
Epoch [10/10]: 100%|██████████| 750/750 [00:28<00:00, 26.37it/s, D_loss=0.75, G_loss=11.9]
Training Complete. Check /results folder for Pix2Pix outputs.
```

```
print("Test images:", len(test_dataset))
e, r = next(iter(test_loader))
print("Edge shape:", e.shape)
print("Real shape:", r.shape)
```

```
Test images: 200
Edge shape: torch.Size([4, 3, 128, 128])
Real shape: torch.Size([4, 3, 128, 128])
```